AEGISSERVE

Secure and Efficient Multi-Agent LLM Inference in Distributed Clouds

AI SYSTEMS	SECURITY	RESILIENCE

WHITE PAPER / RESEARCH PROPOSAL

Version 0.2 - August 2026
Shivam Kumar

Proposed system and benchmark. No measured AegisServe performance is claimed.

Abstract

Multi-agent large language model applications transform one user task into a dependency graph of inference calls. Supervisors trigger parallel workers, agents exchange growing contexts, and aggregators wait on tail requests. These workflows create shared prefixes, bursty arrivals, critical paths, and tenant-specific state that conventional request-level serving policies do not represent. At the same time, the optimizations used to make LLM inference economical - continuous batching, speculative decoding, and KV-cache sharing - interact with latency, fairness, failure recovery, and cross-tenant isolation.

This white paper presents AegisServe, an AI-systems benchmark and serving prototype that models a multi-agent application as a tenant-owned directed acyclic graph. Its executable Workflow-Cache-Fair (WCF) reference policy combines remaining-path criticality, fair-service deficit, predicted cache locality, service cost, failure risk, and endpoint load. The benchmark evaluates six mechanism families: speculative decoding, batching, KV caching, resource scheduling, failure recovery, and multi-tenancy security. It measures request latency and throughput, but makes workflow SLO goodput, cross-tenant cache reuse, timing distinguishability, fairness, and client-visible recovery first-class outcomes. The artifact includes a deterministic DAG trace generator, an OpenAI-compatible replay client, coverage-aware metric aggregation, a cache timing-isolation audit, secret-safe run manifests, reproducible configurations, and strict evidence gates.

The central research question is whether joint workflow and tenant awareness can improve useful serving capacity without exchanging security or resilience for speed. AegisServe is designed to make a negative answer scientifically useful: every claim must be linked to raw events, a run manifest, a fixed configuration, and a stated threat model.

1. Problem statement

Modern LLM serving systems have made individual inference requests substantially more efficient. Iteration-level scheduling allows active batches to change between decode steps [1]. Paged KV memory reduces fragmentation and supports larger dynamic batches [2]. Speculative decoding uses a smaller approximation to propose tokens that a target model verifies in parallel, preserving the target distribution under the algorithm's assumptions [3]. Prefill/decode disaggregation separates compute-heavy prompt processing from memory-bandwidth-heavy generation [4]. Distributed prefix-aware schedulers route requests toward reusable state [5].

Multi-agent applications change the unit of value. A user-visible task may consist of a supervisor call, N worker calls, several debate rounds, tool-mediated continuations, and a final synthesis. The task finishes only when its dependency graph finishes. A request scheduler can increase output tokens per second while delaying an aggregator on the workflow critical path. It can route every request to the hottest cache and starve a tenant whose prompts do not share a prefix. It can recover a failed replica while losing the in-progress stream needed by the parent workflow.

Security also becomes a performance concern. Prefix caching deliberately makes requests with matching prefixes faster. If cache identity is not separated by security domain, a tenant can test candidate prefixes and infer whether another tenant warmed the cache. Partitioning all state avoids that channel but can reduce reuse. Moving KV blocks across nodes to improve utilization or recovery expands the trusted path. The system must quantify the cost of its isolation policy rather than treating security as an unmeasured checkbox.

AegisServe therefore defines the unit of optimization as a completed tenant workflow that meets declared latency, security, fairness, and correctness gates. Raw token throughput remains important, but it is not the objective by itself.

2. System model

An AegisServe workload contains tenants, workflows, agent calls, inference workers, and KV state. Each workflow is a DAG $G = (V, E)$. A vertex v in V is one model call with a prompt, requested output budget, role, tenant identity, security domain, arrival time, and deadline. An edge (u, v) means v cannot be admitted until u produces the required output. The model may be replicated, tensor parallel, pipeline parallel, or separated into prefill and decode pools. These deployment choices are recorded experimental factors.

The design assumes an authenticated gateway binds a request to a server-derived tenant ID. Clients do not choose arbitrary cache domains. KV blocks may occupy GPU HBM, host memory, or a remote store. Each block is identified by at least the model, tokenizer, adapter, cache format, prefix hash chain, and security domain. The serving control

plane can observe workflow dependencies and declared SLOs, but raw production prompts are not required in the benchmark log.

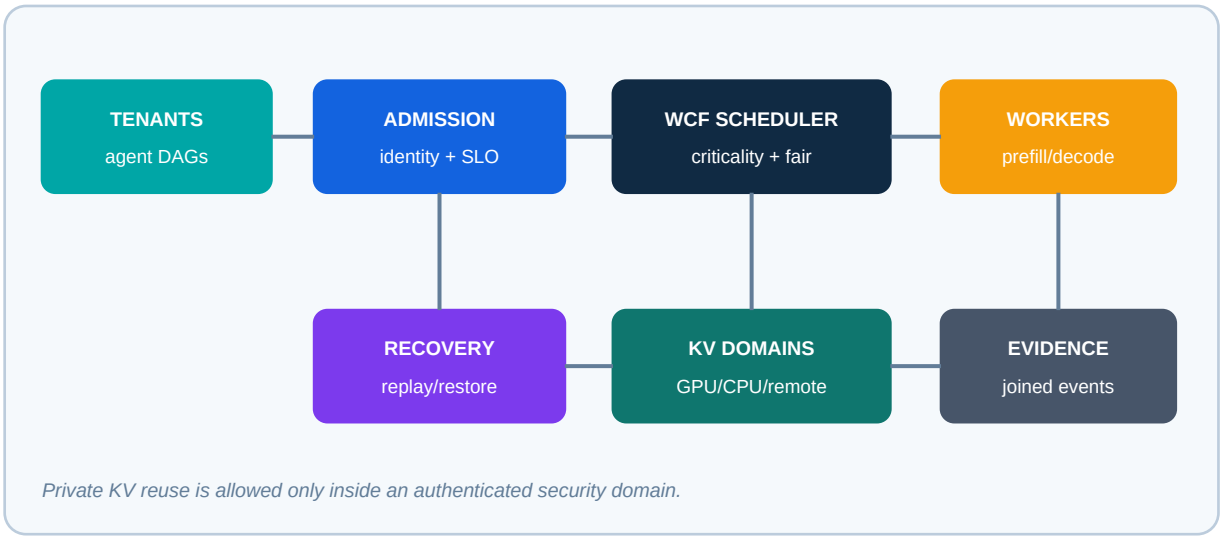

Figure 1. AegisServe logical architecture and evidence path.

Figure 1 shows the logical path. The admission layer releases dependency-ready vertices. The scheduler chooses a worker using workflow criticality, tenant fairness, cache locality, and worker health. The KV manager enforces security-domain lookup rules across tiers. The recovery coordinator chooses recomputation, checkpoint restore, or replica resume after a fault. The evidence plane joins client, engine, cache, and infrastructure events by stable IDs.

3. Related systems and the open gap

3.1 Batching and KV memory

Orca introduced iteration-level scheduling and selective batching for generative models, demonstrating why fixed request batches waste capacity when sequences finish at different times [1]. vLLM's PagedAttention applies paging ideas to the dynamically growing KV cache; its paper reports 2x to 4x throughput improvement over the evaluated baselines at similar latency, with larger gains in memory-intensive regimes [2]. These results establish that batch formation and KV memory management are inseparable.

The AegisServe question is different: which ready agent call should enter the next batch? Sequence length and arrival time are not enough. A short aggregator may unblock a completed workflow, while a cache-rich worker request may save substantial prefill compute. The scheduler must expose this trade-off and measure it at workflow level.

3.2 Speculative decoding under load

Leviathan, Kalman, and Matias formalized speculative decoding with exact sampling from the target model distribution and reported 2x to 3x acceleration for their T5-XXL experiments [3]. A serving deployment adds contention: the draft model consumes compute and memory, acceptance depends on workload and model pairing, and batched verification changes with concurrency. For agent workflows, roles may have different acceptance behavior. Code generation, summarization, planning, and judging should not be assumed to share one optimal speculative-token count.

AegisServe records proposed and accepted draft tokens, target steps, batch pressure, and workflow latency. A high acceptance ratio is not accepted as evidence of a speedup without wall-clock and resource measurements.

3.3 Distributed scheduling and prefill/decode separation

DistServe separates prefill and decode to optimize goodput subject to independent TTFT and TPOT constraints [4]. Preble co-optimizes prefix reuse and load balancing across distributed workers; its evaluation reports large average and tail-latency improvements for the tested models and workloads [5]. Kairos directly studies multi-agent serving and proposes workflow- aware priority scheduling plus memory-aware dispatch, reporting end-to-end latency reductions in its evaluated public-cloud setting [7].

More recent work further narrows the novelty boundary. SAGA studies workflow-atomic scheduling for agent inference on GPU clusters [13]. A policy-driven runtime proposes an agent-aware observe, score, predict, and act layer for cross-cutting serving policies [14]. Cascade applies remaining SLO budget to request scheduling and hierarchical KV management while preserving fairness [15]. Llumnix demonstrates request migration for dynamic LLM serving [16], while FastServe studies iteration-level preemptive scheduling [17]. The first three are recent preprints; their findings require confirmation through released artifacts and independent runs.

These systems motivate the benchmark, but they also mean WCF is not claimed as novel merely for combining workflow, cache, fairness, and SLO signals. AegisServe's narrower contribution is an executable reference policy plus a common evidence contract that joins efficiency, tenant isolation, and client-visible failure recovery. WCF must be compared against workflow-only, cache-only, and fair-only ablations so improvements cannot be attributed vaguely to "awareness."

3.4 Failure recovery

Generative inference is stateful: a failure can discard a large prefill and a growing KV cache. DejaVu proposes KV streaming, microbatch swapping, and state replication for fast, fault-tolerant serving [6]. Ray Serve documents actor replacement and control-state recovery, while also noting that transient router and replica state can be lost when machines fail [10]. Infrastructure recovery therefore does not by itself prove request or workflow recovery.

AegisServe instruments four times: fault injection, fault detection, service readiness, and the first correct post-recovery token. It checks for missing, duplicated, and misattributed output and reports recomputed tokens. A workflow counts as complete only if all required DAG vertices complete correctly.

3.5 Multi-tenant KV security

Current vLLM prefix-cache documentation includes cache salts as part of block identity and recommends cryptographic hashing where collision behavior could leak information in multi-tenant environments [9]. Recent preprints study timing side channels and selective sharing. SafeKV reports a privacy-aware public/private cache design and quantifies a performance-isolation trade-off in its experiments [11]. KVGov proposes principal-keyed salting and a governance layer for multiple timing attacks [12]. These are recent research claims, not assumptions that every engine path is correctly isolated.

AegisServe turns the intended cache separation into a black-box invariant: two otherwise identical requests from distinct security domains must not reuse private KV blocks through any tested API path or cache tier. A positive control uses an intentionally global cache to verify that the timing experiment can detect a known signal. The isolated treatment must produce zero cross-tenant cache hits and an attacker AUC near chance.

4. Proposed design

4.1 Workflow admission

The admission controller validates DAG acyclicity, bounds graph size, attaches the tenant and security domain, and enforces tenant quotas. It computes a remaining critical-path estimate using predicted service times for each vertex. Estimates are updated from recent prefill and decode observations but never overwrite raw measurements.

The controller exposes only dependency-ready calls to the scheduler. This prevents a flat request queue from admitting downstream prompts before their parents finish and makes burst amplification explicit: completing one supervisor may release many workers at once.

4.2 Workflow-Cache-Fair scheduling

For a ready request r and target worker w, WCF computes:

```
WCF(r, w) = alpha * criticality(r)
          + beta  * tenant_deficit(r)
          + gamma * cache_benefit(r, w)
          - delta * service_cost(r, w)
          - epsilon * failure_risk(w)
          - zeta * endpoint_load(w)
```

The reference implementation uses only trace metadata and prior client observations. Criticality combines remaining-path work, deadline urgency, fanout, and a terminal-node bonus. Tenant deficit is the gap between equal-share admitted work and the tenant's admitted service. Cache benefit is the fraction of prompt tokens predicted to share a prefix already resident at the endpoint, never across security domains. Service cost is requested prompt plus output tokens. Failure risk is the observed endpoint failure rate, and endpoint load is the fraction of client permits currently occupied.

The score is intentionally decomposable. The evaluation removes each term, sweeps weights, and compares with round robin, tenant affinity, cache affinity, and shortest predicted remaining time. If WCF improves only because it receives a larger effective batch or a warmer cache, the ablations will expose that confound.

This WCF implementation controls dependency-ready client admission and endpoint selection. It does not inspect or override an inference engine's internal continuous batch. Engine queueing, preemption, batch composition, and physical KV placement require adapters and must not be inferred from client scheduling decisions.

4.3 Adaptive speculation and batching

WCF does not require speculative decoding. When enabled, an admission policy selects a speculative configuration by agent role, output-length estimate, recent acceptance, and current draft/target pressure. The system may disable speculation when verification reduces batch efficiency or when short outputs cannot amortize draft overhead.

Batching remains engine-owned, but the scheduler supplies priorities and token budgets. Metrics separate client queue delay, engine queue delay, prefill, first token, and decode. This avoids crediting speculation for time saved by a simultaneous batch-policy change.

4.4 Tenant-scoped KV tiers

Private KV blocks use a cache namespace derived from a server secret and tenant identity, for example HMAC(secret, tenant_id). A client cannot provide another tenant's salt. The namespace is included consistently in local and remote cache keys, migration records, and recovery checkpoints.

An optional public domain permits reuse only for content explicitly classified and admitted as shareable, such as a published system template. Private-to-public promotion is denied by default. Remote movement uses authenticated encryption, integrity metadata, expiration, and ownership verification at restore time.

The evaluation compares a global cache, strict per-tenant partition, keyed namespace, and optional public/private design. It reports the security outcome and the lost cache benefit.

4.5 Recovery coordinator

The recovery modes are:

1. Full recompute: retry the prompt on a healthy worker.
2. Metadata checkpoint: preserve request and deterministic replay state, then recompute KV.
3. Incremental KV checkpoint: restore verified recent blocks and recompute the suffix.
4. Replicated KV: resume from a synchronized secondary state.

The most stateful mode is not always best. Checkpoint traffic can compete with decode and remote transfer may exceed recomputation time for short prompts. WCF includes worker and failure-domain health so it can avoid concentrating all ready vertices and their state on one node.

5. Benchmark methodology

5.1 Workload topology

The trace generator produces three DAG families. A chain stresses critical-path and session affinity. A supervisor graph creates one root, parallel workers, and a fan-in aggregator. A debate graph synchronizes several agents across rounds and ends with a judge. Each trace fixes tenant, role, dependencies, requested prompt/output size, shared-prefix size, arrival, and deadline.

The systems study replays synthetic deterministic prompts so scheduler effects are not confounded by model decisions. A separate quality study runs real coding, retrieval, and deliberation tasks. The same systems policy must preserve model, tokenizer, sampling, and task scoring across treatments.

5.2 Factor families

Mechanism	Controlled treatments	Required outcomes
Speculation	off; draft; 3/5/8 tokens	acceptance; TPOT; quality
Batching	concurrency; token budget	queue; TTFT; goodput
KV cache	off; local; tenant; remote	hits; transfer; memory
Scheduling	RR; affinity; WCF	workflow tail; fairness
Recovery	recompute; checkpoint; replica	RTO; lost work; correctness
Security	global; partition; keyed salt	cross-hits; AUC; overhead

Table 1. Required mechanism families and outcomes.

The first study uses a two-level fractional factorial design to screen the six mechanism families. Focused studies then expand important interactions: speculation by batching and length; cache reuse by isolation and scheduler; failure type by recovery mode; and tenant skew by fairness policy. At least five independent repetitions are required for screening and ten for final tail results. Treatment order is randomized within identical hardware blocks.

5.3 Metrics

Time to first token (TTFT) is measured from client send to first streamed output. Time per output token (TPOT) excludes the first token and divides remaining request time by remaining output tokens. vLLM's benchmark documentation notes that speculative decoding can return multiple tokens in one streamed chunk, so inter-token latency and TPOT are not equivalent [8]. AegisServe records the formula and measurement point rather than relying on ambiguous metric names.

Workflow latency is the final required vertex completion minus workflow arrival. Workflow SLO goodput is the number of correctly completed workflows meeting all declared constraints per second. Other outcomes include output token throughput, GPU utilization, batch occupancy, actual cached tokens, cache transfers, draft acceptance, recomputed work, recovery time, Jain fairness, cross-tenant hits, and timing attack AUC.

An unavailable engine signal is null, not zero. Every summary reports observation coverage for usage, cache, speculation, and security evidence. A successful workflow is SLO-evaluable only when every required request reports enough timing and token evidence to test its declared TTFT and TPOT constraints. Failed workflows remain known SLO failures.

5.4 Validity and statistics

Each run writes a sidecar manifest containing a run identifier, configuration digest, Git revision and dirty state, package and Python versions, seed, trace hash, event-log hash, model and endpoint identities, treatment allowlist, timestamps, and coverage counts. It does not contain environment values or prompt bodies. Model/tokenizer revisions, engine/container version, GPU and network inventory, engine counters, and infrastructure metrics must be added by the cluster adapter before a hardware performance claim is valid. Fault runs include the exact target and timeline. Failed requests remain in all completion and SLO denominators.

Results are summarized across independent runs using medians and bootstrap 95 percent confidence intervals. Per-request samples are not treated as independent run replicates. Comparisons use paired traces and seeds. Cache treatments reset state between runs unless the initial state is explicitly controlled.

6. Security evaluation

The primary adversary is an authenticated malicious tenant that can send adaptive prompts, create load, and measure its own latency. It cannot administer the cluster, read host memory, or break authenticated encryption. In-scope outcomes are prefix/KV timing leakage, cross-tenant state reuse, starvation, noisy-neighbor interference, unprotected KV movement, and stale or misattributed recovery.

The cache experiment contains four arms:

1. Cold control: no tenant has warmed the candidate prefix.
2. Positive control: tenant A warms an intentionally global cache and tenant B probes.
3. Isolated treatment: tenant A warms its private domain and tenant B probes.
4. Same-tenant utility: tenant A warms and reuses its own private prefix.

The positive control must show a detectable effect before a negative treatment is accepted. The default engineering gate requires zero cross-tenant cached-token hits and attacker ROC AUC no greater than 0.60. This threshold is not a formal proof of non-interference; the report includes sample size, confidence interval, network conditions, and statistical power. Security cost is reported as changes in TTFT, throughput, cache memory, and workflow SLO goodput.

The current scope excludes a malicious hypervisor, physical side channels, model-level prompt injection, unsafe tool execution, training-data extraction, and formal verification of GPU kernels. These exclusions limit the claim; they do not imply those risks are solved.

7. Failure evaluation

The fault matrix covers inference-process crash, GPU worker crash, node loss, and injected network delay. Faults are injected at controlled prefill and decode progress points. Each fault treatment records detection latency, service restoration, first correct post-recovery token, recomputed tokens, checkpoint/replica bytes, duplicate or missing output, request success, workflow completion, and post-fault SLO attainment.

A health endpoint or restarted actor is insufficient evidence. The client stream is the authoritative continuity observation. For deterministic sampling runs, recovered output is compared with an unfaulted control. For stochastic runs, protocol correctness and token stream structure are verified without demanding byte-identical text.

The recovery hypothesis is conditional: incremental KV restore should help only when saved prefill/decode work exceeds checkpoint, transfer, and validation overhead. The experiment therefore sweeps prompt length, failure phase, cache size, and network bandwidth rather than reporting one favorable point.

8. Artifact implementation

The v0.2 artifact implements strict YAML and trace validation, stable configuration digests, deterministic DAG trace generation, Poisson and bursty arrivals, uniform and Zipf tenants, dependency-aware asynchronous replay, round-robin and tenant-affinity baselines, executable WCF client admission, tenant HMAC cache salts, coverage-aware request and workflow metrics, strict SLO goodput, Jain fairness, a cache timing audit, and hashed run manifests.

The live runner targets OpenAI-compatible chat-completion endpoints. It records client-side TTFT and completion times and consumes reported usage/cached-token fields where the engine provides them. Unreported signals remain null and reduce the corresponding coverage count. Speculation counters, GPU telemetry, cache-transfer metrics, inference-engine batch control, and destructive fault injection require deployment-specific adapters. They are specified but not falsely simulated in the live runner.

This separation matters for research integrity. A client can prove request timing and DAG completion. It cannot prove GPU batch composition, KV migration, or worker recovery without server and infrastructure evidence. The repository labels proposed, simulated, measured, derived, and prior-work claims separately.

9. Expected contributions and falsification

A successful study would contribute: a multi-agent serving workload model that retains DAG and prefix structure; a benchmark joining efficiency with isolation and recovery; an executable WCF reference treatment with component ablations; and a reproducible, coverage-aware evidence contract for repeated cloud runs. It does not claim that combining workflow, cache, fairness, or SLO signals is independently novel.

The principal hypotheses are:

1. H1: WCF reduces p95 workflow latency under supervisor and debate loads at equal hardware.
1. H2: the best speculation setting changes with batch pressure and agent role.
1. H3: keyed tenant namespaces eliminate cross-tenant hits with bounded efficiency cost.
1. H4: incremental KV recovery beats recompute only beyond a measurable state-size threshold.
1. H5: token-throughput optimization alone produces worse workflow SLO or fairness outcomes.

Each can fail. WCF may add overhead without improving latency. Tenant partitioning may be cheap enough that selective sharing is unnecessary. KV restore may lose to fast recompute on modern accelerators. Speculation may regress under the studied concurrency. The artifact is designed to retain and explain these negative results.

10. Limitations and responsible use

Synthetic token targets approximate but do not guarantee tokenizer-exact lengths; measured engine usage is authoritative. Trace replay cannot reproduce every tool delay, cancellation, or semantic branch of a real agent application. The initial model sizes may not generalize to large mixture-of-experts systems. Public cloud noise can widen tails, and two-zone tests do not represent global multi-region deployment.

Timing experiments must use synthetic secrets owned by the researcher. AegisServe is not a tool for probing third-party tenants or services. Fault injection must run only in dedicated authorized infrastructure. Raw prompts are excluded from default logs, and experiment secrets are supplied through the environment rather than committed configuration.

11. Conclusion

Multi-agent inference is not merely more requests. It is a stateful, dependent, bursty, and multi-tenant systems workload whose useful output is a completed workflow. The same KV cache that removes prefill work can create a timing signal; the same batching policy that maximizes tokens can delay a critical aggregator; the same worker restart that restores capacity can lose the workflow's active state.

AegisServe proposes one integrated way to study these tensions. Its benchmark makes six mechanism families comparable under a common DAG workload, evidence schema, and validity policy. Its executable WCF reference treatment combines criticality, fairness, predicted cache locality, service cost, failure risk, and endpoint load at the client-admission boundary. Most importantly, its claims are conditional on reproducible, coverage-aware measurement. The next step is not to declare the joint policy faster or safer, but to implement the cluster adapters, execute the preregistered matrix, and publish the raw evidence alongside the conclusions.

References

[1] G.-I. Yu et al. "Orca: A Distributed Serving System for Transformer-Based Generative Models." OSDI 2022. https://www.usenix.org/conference/osdi22/presentation/yu

[2] W. Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP 2023. https://arxiv.org/abs/2309.06180

[3] Y. Leviathan, M. Kalman, and Y. Matias. "Fast Inference from Transformers via Speculative Decoding." ICML 2023. https://proceedings.mlr.press/v202/leviathan23a.html

[4] Y. Zhong et al. "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving." OSDI 2024. https://arxiv.org/abs/2401.09670

[5] V. Srivatsa et al. "Preble: Efficient Distributed Prompt Scheduling for LLM Serving." Preprint, 2024. https://arxiv.org/abs/2407.00023

[6] F. Strati et al. "DejaVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving." 2024. https://arxiv.org/abs/2403.01876

[7] J. Chen et al. "Kairos: Low-latency Multi-Agent Serving with Shared LLMs and Excessive Loads in the Public Cloud." 2025. https://arxiv.org/abs/2508.06948

[8] vLLM Project. "Benchmark CLI: Understanding the Latency Metrics." Accessed August 2026. https://docs.vllm.ai/en/latest/benchmarking/cli/

[9] vLLM Project. "Automatic Prefix Caching." Accessed August 2026. https://docs.vllm.ai/en/latest/design/prefix_caching/

[10] Ray Project. "Ray Serve Architecture and Fault Tolerance." Accessed August 2026. https://docs.ray.io/en/latest/serve/architecture.html

[11] K. Chu et al. "Selective KV-Cache Sharing to Mitigate Timing Side-Channels in LLM Inference." Preprint, 2025. https://arxiv.org/abs/2508.08438

[12] T. C. Addagada. "Governing the KV Cache: Preventing Timing Side-Channel Leakage in Multi-Tenant LLM Inference." Preprint, 2026. https://arxiv.org/abs/2608.09225

[13] D. Guo, J. Wu, and S. M. Yiu. "SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters." Preprint, 2026. https://arxiv.org/abs/2605.00528

[14] R. Zhang, C. Kim, and L. Hu. "A Policy-Driven Runtime Layer for Agentic LLM Serving." Preprint, 2026. https://arxiv.org/abs/2605.27744

[15] M. Adnan et al. "Cascade: Exploiting SLO-Aware Latency Budget for Fair and High Goodput LLM Inference Serving." Preprint, 2026. https://arxiv.org/abs/2608.06557

[16] B. Sun et al. "Llumnix: Dynamic Scheduling for Large Language Model Serving." OSDI 2024. https://www.usenix.org/conference/osdi24/presentation/sun-biao

[17] B. Wu et al. "FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference." NSDI 2026. https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang